ORM, generación de reportes y consumo de servicios web

UNICAM – CUR Matagalpa – Departamento de Ciencias, Tecnología y Salud

Fecha: octubre, 2026

Universidad Nacional Autónoma de Nicaragua, Managua · Centro Universitario Regional de Matagalpa – Departamento de Ciencias, Tecnología y Salud

Manual didáctico Encuentro N°3: ORM, generación de reportes y consumo de servicios web

Autor: Heydy Judelki Mairena Palacios

Componente: Programación de aplicaciones de escritorio

Carrera: Ingeniería de Sistemas

Año: II año – II semestre

Octubre, 2026

¡Universidad del Pueblo y para el Pueblo!

Elaborado por:	**Revisado por:**	**Autorizado por:**
Heydy Judelki Mairena Palacios	Erick Lanzas	Erick Lanzas

Presentación del material

Este módulo cierra el proyecto integrador. AgroStock dejará de escribir SQL a mano gracias a un ORM, generará reportes de inventario en PDF y Excel, y obtendrá productos de prueba desde un servicio web (Mockaroo) para probar el inventario con cientos de datos. Se abordan las Unidades IV y V del programa y se reúnen los aprendizajes del semestre.

Proyecto integrador: AgroStock: aplicación de escritorio para el inventario de un agroservicio o cooperativa de la zona (productos, categorías, existencias, movimientos y reportes). Se construye por etapas: v1 interfaz gráfica (E1) → v2 base de datos SQLite + importación concurrente (E2) → v3 ORM, reportes PDF/Excel y consumo de servicio web (E3).

Objetivo de aprendizaje del encuentro

- Utilizar un ORM (Entity Framework Core) para realizar operaciones CRUD sin escribir SQL manualmente y generar reportes en PDF y Excel a partir de datos reales.
- Consumir servicios web RESTful desde una aplicación de escritorio, procesando respuestas JSON o XML y manejando fallos de conexión.

Contenidos de este encuentro

- **Unidad IV. Uso de ORM y generación de reportes:** Uso de ORM para realizar operaciones CRUD sobre una base de datos; Uso de herramienta para la generación de reportes.
- **Unidad V. Consumo de servicios web:** Clases para el consumo de servicios web; Estudios de caso para el consumo de servicios web.

A. Entrada temática

Vivencia

La cooperativa a la que pertenece el agroservicio de doña Martha pide cada lunes un reporte del inventario en PDF y una copia en Excel. Su hijo copia a mano la tabla de AgroStock en Excel y suma los valores con la calculadora del celular. Además, para probar el programa con muchos productos tuvo que inventarlos uno por uno.

Antes de continuar, reflexione:

- ¿Qué partes de este proceso podría automatizar AgroStock?
- En AgroStock v2 escribió muchas sentencias SQL parecidas. ¿Qué pasaría si mañana hubiera que cambiar a SQL Server?
- ¿De dónde podrían salir cientos de productos de prueba realistas sin escribirlos a mano?
- ¿Cuánto papel y cuántos viajes se ahorran si el reporte se envía en digital?

Responda las preguntas anteriores en su cuaderno y compártalas con su pareja.

Actividad SQA: en su cuaderno o en el móvil, elabore una tabla de tres columnas —Qué sé, Qué quiero saber, Qué aprendí— sobre ORM, generación de reportes y servicios web. Complete únicamente las dos primeras columnas por ahora; la tercera la completará al cierre de este material.

Lo que esperamos aprender: manipular la base de datos con objetos (ORM), producir reportes profesionales y conectar la aplicación con servicios en Internet de forma robusta; y lo aplicará al caso de doña Martha.

B. Desarrollo central

Fundamentación científica

Unidad IV · Uso de ORM y generación de reportes

Unidad IV · 1. ¿Qué es un ORM?

Un ORM (Object-Relational Mapper, mapeador objeto-relacional) traduce automáticamente entre los objetos del programa y las tablas de la base de datos. El programador trabaja con clases y colecciones; el ORM genera el SQL, abre las conexiones y convierte las filas en objetos. En .NET, el ORM oficial es Entity Framework Core (EF Core).

Mundo orientado a objetos	Mundo relacional
Clase Producto	Tabla productos
Propiedad Nombre	Columna Nombre
Objeto (instancia)	Fila
Referencia Producto.Categoria	Clave foránea CategoriaId
HasOne(...).WithMany()	Relación 1:N entre categorias y productos

- Ventajas: menos código repetitivo, consultas verificadas por el compilador (LINQ), independencia del motor (SQLite, SQL Server, MySQL, PostgreSQL), protección contra inyección SQL.
- Límites: menos control sobre el SQL generado; consultas muy complejas pueden ser menos eficientes; hay que entender qué consulta se ejecuta y cuándo.

Unidad IV · 2. Entity Framework Core: entidades y DbContext

En AgroStock v3 el modelo de datos se describe con clases: tres entidades (Producto, Categoria y Movimiento) y un DbContext (AgroStockContext) que reemplaza al script agrostock.sql del Encuentro 2.

```
public class Producto
{
    public int Id { get; set; }                         // clave primaria (por convención)
    public string Codigo { get; set; } = "";
    public string Nombre { get; set; } = "";
    public int CategoriaId { get; set; }                // clave foránea
```

```
    public Categoria? Categoria { get; set; }           // propiedad de navegación
    public decimal Precio { get; set; }
    public int Existencia { get; set; }
    public DateTime? Vencimiento { get; set; }
    public string CategoriaNombre => Categoria?.Nombre ?? "";   // solo lectura, para la
cuadrícula
}

public class Categoria { public int Id { get; set; } public string Nombre { get; set; } = "";
}

public class Movimiento
{
    public int Id { get; set; }
    public int ProductoId { get; set; }
    public Producto? Producto { get; set; }
    public string Tipo { get; set; } = "ENTRADA";       // ENTRADA o SALIDA
    public int Cantidad { get; set; }
    public DateTime Fecha { get; set; } = DateTime.Today;
}

public class AgroStockContext : DbContext
{
    public DbSet<Categoria> Categorias => Set<Categoria>();
    public DbSet<Producto> Productos => Set<Producto>();
    public DbSet<Movimiento> Movimientos => Set<Movimiento>();

    protected override void OnConfiguring(DbContextOptionsBuilder options)
        => options.UseSqlite($"Data Source={Ruta}");          // archivo agrostock.db

    protected override void OnModelCreating(ModelBuilder mb)
    {
        mb.Entity<Producto>(e =>
        {
            e.ToTable("productos", t => t.HasCheckConstraint("ck_precio", "precio > 0"));
            e.Property(p => p.Codigo).IsRequired();
            e.HasIndex(p => p.Codigo).IsUnique();             // código sin repetir
            e.HasOne(p => p.Categoria).WithMany()
             .HasForeignKey(p => p.CategoriaId).OnDelete(DeleteBehavior.Restrict);
        });

        mb.Entity<Categoria>().HasData(                       // datos iniciales
            new Categoria { Id = 1, Nombre = "Fertilizantes" },
            new Categoria { Id = 2, Nombre = "Semillas" });   // ... y el resto
    }
}

public static class Db
{
    public static void Inicializar()
    {
        using var ctx = new AgroStockContext();
```

```
        ctx.Database.EnsureCreated();   // crea agrostock.db, tablas, restricciones y datos
iniciales
    }
}
```

El DbContext representa una sesión de trabajo con la base: rastrea los objetos que se agregan o modifican y, al llamar a SaveChanges(), genera y ejecuta los INSERT, UPDATE y DELETE necesarios dentro de una transacción. Database.EnsureCreated() crea la base y las tablas si no existen; en proyectos mayores se usan migraciones (dotnet ef migrations add).

Importante. EnsureCreated no actualiza una base que ya tiene tablas: si cambia el modelo, borre agrostock.db de la carpeta bin\Debug\net8.0-windows y ejecute de nuevo. La base se crea sola la primera vez, con las 4 categorías, los 14 productos y los 6 movimientos iniciales.

Del SQL del Encuentro 2 al modelo en C#:

En agrostock.sql (E2)	En el modelo (E3)
id INTEGER PRIMARY KEY AUTOINCREMENT	Propiedad Id (por convención es la clave primaria)
NOT NULL	IsRequired() o un tipo que no admite null
UNIQUE	HasIndex(...).IsUnique()
CHECK (precio > 0)	HasCheckConstraint("ck_precio", "precio > 0")
REFERENCES categorias(id)	HasOne(...).WithMany().HasForeignKey(...).OnDelete(Restrict)
Los INSERT de ejemplo	HasData(...)
DEFAULT date('now', 'localtime')	Fecha = DateTime.Today (se calcula en C#)

Unidad IV · 3. CRUD con LINQ sin escribir SQL

Operación	EF Core	SQL generado (aprox.)
Read	ctx.Productos.Include(p => p.Categoria).Where(...).OrderBy(p => p.Nombre).ToList()	SELECT … FROM productos JOIN categorias … WHERE … ORDER BY nombre
Create	ctx.Productos.Add(p); ctx.SaveChanges();	INSERT INTO productos …
Update	ctx.Productos.Update(p); ctx.SaveChanges();	UPDATE productos SET … WHERE Id = …

Operación	EF Core	SQL generado (aprox.)
Delete	ctx.Productos.Remove(p); ctx.SaveChanges();	DELETE FROM productos WHERE Id = …

ProductoRepository conserva los mismos métodos del Encuentro 2, pero cada uno abre un AgroStockContext con using y no contiene ninguna sentencia SQL:

```
public List<Producto> Listar(string filtro = "")
{
    using var ctx = new AgroStockContext();
    return ctx.Productos
        .Include(p => p.Categoria)                                  // JOIN con categorias
        .Where(p => EF.Functions.Like(p.Nombre, $"%{filtro}%")
                 || EF.Functions.Like(p.Codigo, $"%{filtro}%"))     // LIKE con parámetro
        .OrderBy(p => p.Nombre)
        .ToList();
}

public void RegistrarMovimiento(int productoId, string tipo, int cantidad)
{
    using var ctx = new AgroStockContext();
    var p = ctx.Productos.Find(productoId) ?? throw new InvalidOperationException("Producto
no encontrado.");
    if (tipo == "SALIDA" && p.Existencia < cantidad)
        throw new InvalidOperationException("Existencia insuficiente: no se registró nada.");

    p.Existencia += tipo == "ENTRADA" ? cantidad : -cantidad;
    ctx.Movimientos.Add(new Movimiento { ProductoId = productoId, Tipo = tipo,
                                         Cantidad = cantidad, Fecha = DateTime.Today });
    ctx.SaveChanges();   // una sola transacción para ambos cambios
}
```

Ejecución diferida: una consulta LINQ no va a la base hasta que se enumera (ToList, First, Count, foreach). Construye la consulta completa antes de materializarla.

InsertarVarios. inserta muchos productos con un solo SaveChanges (una transacción) y devuelve los códigos que ya existían en lugar de fallar; Limpio() copia el producto sin su objeto Categoria para que EF no intente actualizar también esa tabla.

Errores del ORM. cuando la base rechaza un cambio, EF Core lanza una DbUpdateException; la causa real está en ex.InnerException (la SqliteException). FrmPrincipal la evalúa con when para avisar de un código repetido o de un producto con movimientos sin cerrar el programa:

```
catch (DbUpdateException ex) when (ex.InnerException is SqliteException {
SqliteExtendedErrorCode: 2067 })
{   // UNIQUE: el código ya existe
```

```
    MessageBox.Show(this, $"El código {codigo} ya existe.", "No se guardó");
}
catch (DbUpdateException ex) when (ex.InnerException is SqliteException { SqliteErrorCode: 19
} fk
                                    && fk.Message.Contains("FOREIGN KEY"))
{   // el producto tiene movimientos registrados
    MessageBox.Show(this, "No se puede eliminar: el producto tiene movimientos registrados.",
"No se eliminó");
}
```

Unidad IV · 4. Generación de reportes

Un reporte presenta datos de la base de forma organizada para tomar decisiones. Su estructura clásica: encabezado (título, fecha, filtros), detalle (filas), agrupaciones y totales, y pie (numeración). El programa pide exportar a PDF o Excel; usamos dos bibliotecas libres para uso académico: QuestPDF (PDF con API fluida en C#) y ClosedXML (Excel sin necesidad de tener Office instalado).

ReporteService recibe la lista de productos ya leída y la ruta que eligió el usuario. El PDF usa A4 horizontal, resalta las existencias menores que 10 y muestra en el pie el total y la numeración de páginas:

```
Document.Create(doc => doc.Page(page =>
{
    page.Size(PageSizes.A4.Landscape());
    page.Header().Column(col =>
    {
        col.Item().Text("AgroStock - Reporte de inventario").FontSize(18).Bold();
        col.Item().Text($"Generado: {DateTime.Now:dd/MM/yyyy HH:mm}").FontSize(9);
    });
    page.Content().Table(tabla =>
    {
        // ... columnas y encabezado de la tabla ...
        foreach (var p in productos)
        {
            var fondo = p.Existencia < 10 ? Colors.Red.Lighten4 : Colors.White;   //
existencia baja resaltada
            Celda(tabla, p.Codigo, fondo);
            Celda(tabla, p.Nombre, fondo);
            Celda(tabla, p.Precio.ToString("N2"), fondo);   // ... y las demás columnas
        }
    });
    page.Footer().Row(r =>
    {
        r.RelativeItem().Text($"Productos: {productos.Count}   |   Valor del inventario: C$
{valorTotal:N2}");
        r.RelativeItem().AlignRight().Text(t =>
        { t.Span("Página "); t.CurrentPageNumber(); t.Span(" de "); t.TotalPages(); });
```

```
    });
})).GeneratePdf(ruta);
```

El libro de Excel escribe el valor de cada fila y el total como fórmulas, de modo que se recalculen si el usuario cambia un dato:

```
using var libro = new XLWorkbook();
var hoja = libro.Worksheets.Add("Inventario");
// ... encabezados con estilo (negrita, fondo verde oscuro) ...
int fila = 2;
foreach (var p in productos)
{
    hoja.Cell(fila, 1).Value = p.Codigo;
    hoja.Cell(fila, 4).Value = p.Precio;
    hoja.Cell(fila, 5).Value = p.Existencia;
    hoja.Cell(fila, 7).FormulaA1 = $"D{fila}*E{fila}";          // valor = precio por
existencia
    fila++;
}
hoja.Cell(fila, 7).FormulaA1 = $"SUM(G2:G{fila - 1})";           // total como fórmula
hoja.Range(2, 4, fila, 4).Style.NumberFormat.Format = "#,##0.00";
hoja.SheetView.FreezeRows(1);
libro.SaveAs(ruta);
```

Buenas prácticas: declarar la licencia de QuestPDF (QuestPDF.Settings.License = LicenseType.Community) en el constructor estático, pedir la ruta con SaveFileDialog (Unidad I), capturar IOException cuando el archivo está abierto en otro programa, usar fórmulas en Excel en lugar de valores calculados y mantener el reporte en una clase de servicio que no conoce los formularios.

Unidad V · Consumo de servicios web

Unidad V · 5. Servicios web y HTTP

Un servicio web es una aplicación que ofrece datos o funciones a otras aplicaciones a través de la red usando el protocolo HTTP. En el estilo REST, cada recurso tiene una URL y se opera con métodos HTTP; las respuestas suelen venir en JSON (y en servicios más antiguos, en XML).

Elemento	Ejemplo
Método	GET (leer), POST (crear), PUT (reemplazar), DELETE (eliminar); AgroStock usa GET
URL	https://api.mockaroo.com/api/generate.json?schema=agrostock_productos&count=20&array=true

Elemento	Ejemplo
Código de estado	200 OK · 401 Clave incorrecta · 404 No encontrado (el schema no existe) · 500 Error del servidor
Encabezados	X-API-Key: (su clave de API)
Cuerpo (JSON)	[{ "codigo": "SEM-4821", "nombre": "Semilla de arroz (libra)", "categoria": "Semillas", "precio": 85.50, "existencia": 40, "vencimiento": "2027-05-01" }]

Unidad V · 6. Clases para el consumo de servicios web

Clase	Función
HttpClient	Envía peticiones HTTP y recibe respuestas. Se crea una sola instancia (static readonly) para toda la aplicación; AgroStock fija un tiempo máximo de 30 segundos.
HttpRequestMessage	Describe la petición: método GET, URL y encabezados (X-API-Key).
HttpResponseMessage	Respuesta: StatusCode, IsSuccessStatusCode y contenido (ReadAsStream).
System.Text.Json / JsonSerializer.Deserialize<T>	Convierte JSON en objetos C# (deserialización).
[JsonPropertyName]	Relaciona el nombre del campo JSON con la propiedad C#.

La configuración del servicio (dirección, clave y nombre del schema) vive en appsettings.json, que se copia junto al ejecutable; MockarooClient la lee al crearse:

```
{
  "Mockaroo": {
    "BaseUrl": "https://api.mockaroo.com/api/generate.json",
    "ApiKey": "<su clave de API>",
    "Schema": "agrostock_productos"
  }
}

public class ProductoMockaroo
{
    [JsonPropertyName("codigo")]      public string Codigo { get; set; } = "";
    [JsonPropertyName("nombre")]      public string Nombre { get; set; } = "";
    [JsonPropertyName("categoria")]   public string Categoria { get; set; } = "";
    [JsonPropertyName("precio")]      public decimal Precio { get; set; }
    [JsonPropertyName("existencia")]  public int Existencia { get; set; }
    [JsonPropertyName("vencimiento")] public string? Vencimiento { get; set; }
}

private static readonly HttpClient Http = new() { Timeout = TimeSpan.FromSeconds(30) };

public List<ProductoMockaroo> ObtenerProductos(int cantidad)
```

```
{
    var url = $"{baseUrl}?schema={Uri.EscapeDataString(schema)}&count={cantidad}&array=true";
    using var req = new HttpRequestMessage(HttpMethod.Get, url);
    req.Headers.Add("X-API-Key", apiKey);                 // la clave viaja en un encabezado

    using var resp = Http.Send(req);                      // llamada síncrona: la interfaz
espera
    using var flujo = resp.Content.ReadAsStream();
    if (!resp.IsSuccessStatusCode)
    {
        using var lector = new StreamReader(flujo);
        throw new HttpRequestException($"Mockaroo respondió {(int)resp.StatusCode}:
{lector.ReadToEnd()}");
    }
    return JsonSerializer.Deserialize<List<ProductoMockaroo>>(flujo) ?? new();   // JSON a
objetos
}
```

MockarooClient.Convertir descarta las filas inválidas (categoría desconocida, código o nombre de menos de 3 caracteres, precio menor o igual que cero, existencia negativa) para contarlas como rechazadas antes de guardar.

Sin hilos, a propósito. Http.Send es una llamada síncrona: mientras llega la respuesta, la ventana no responde un instante (el cursor cambia a espera). En el Encuentro 2 resolvió ese problema con Task.Run e IProgress; combinar ambas ideas es el reto 3. Siempre se deben prever los fallos: sin conexión (HttpRequestException), tiempo agotado (TaskCanceledException), respuesta inesperada (JsonException) y clave faltante (InvalidOperationException).

```
catch (DbUpdateException ex)
{   // la base rechazó algo: se revirtió toda la importación
    lblEstado.Text = "Se revirtió la importación: " + (ex.InnerException?.Message ??
ex.Message);
}
catch (Exception ex) when (ex is IOException or HttpRequestException or
InvalidOperationException
                               or TaskCanceledException or System.Text.Json.JsonException)
{
    lblEstado.Text = "No se pudo completar: " + ex.Message;
}
```

Unidad V · 7. Estudio de caso: productos de prueba con Mockaroo

Probar un inventario con 14 productos no basta: doña Martha necesita ver cómo se comporta AgroStock con cientos. Mockaroo es un servicio web que genera datos de prueba realistas según un schema; el schema agrostock_productos tiene seis campos (codigo, nombre, categoria, precio, existencia y vencimiento) con los mismos nombres que las propiedades de ProductoMockaroo.

FrmImportar ofrece dos orígenes: el archivo productos_importar.csv y el botón «Obtener de Mockaroo» (de 1 a 1000 productos). Ambos pasan por el mismo método Ejecutar, que guarda con InsertarVarios en una sola transacción: si la base rechaza algo se revierte toda la importación. Al terminar informa cuántos productos entraron, cuántas filas eran inválidas y cuántos códigos estaban repetidos; importar el mismo CSV dos veces no falla, la segunda vez informa los repetidos.

- Analice el servicio antes de usarlo: documentación, límites del plan gratuito y términos de uso.
- No publique su clave: appsettings.json contiene un secreto. Si sube el proyecto a un repositorio público, quítela o cámbiela; en la entrega use la clave que indique su docente.
- Otros casos posibles: consultar el tipo de cambio oficial que publica el Banco Central de Nicaragua (BCN), si ofrece un servicio consultable, o el clima para planificar la aplicación de fertilizantes.

8. Integración: AgroStock v3

AgroStock v3 se construye sobre una copia de la v2 corregida (AgroStock.E03). El código completo de cada archivo, en versión comentada y limpia, está en la guía interactiva Guia_AgroStock_E03. Su estructura es la siguiente:

```
AgroStock.E03/
├── AgroStock.E03.csproj          ← paquetes: EF Core Sqlite, ClosedXML, QuestPDF
├── appsettings.json              ← configuración de Mockaroo (BaseUrl, ApiKey, Schema)
├── productos_importar.csv
├── Program.cs
├── Models/Producto.cs            ← Producto, Categoria y Movimiento
├── Data/AgroStockContext.cs      ← ORM: DbSet, restricciones, relaciones, HasData
├── Data/Db.cs                    ← EnsureCreated()
├── Data/ProductoRepository.cs    ← CRUD con LINQ y SaveChanges
├── Services/ReporteService.cs    ← QuestPDF (PDF) y ClosedXML (Excel)
├── Services/MockarooClient.cs    ← HttpClient + JSON
└── Forms/                        ← FrmPrincipal · FrmProducto · FrmMovimiento
                                     FrmMovimientos · FrmImportar
```

Aspecto	AgroStock v2 (E2)	AgroStock v3 (E3)
Acceso a datos	SQL escrito a mano con Microsoft.Data.Sqlite	LINQ con Entity Framework Core, sin SQL a mano
Esquema de la base	Script agrostock.sql	Modelo en C# (AgroStockContext) con datos iniciales; la base se crea sola

Aspecto	AgroStock v2 (E2)	AgroStock v3 (E3)
Reportes	No tiene	PDF (QuestPDF) y Excel (ClosedXML) del inventario
Datos de prueba	Archivo productos_importar.csv	CSV o servicio web Mockaroo
Importación	Task.Run, IProgress y cancelación con rollback	Síncrona, sin hilos, en una sola transacción; informa inválidos y repetidos
Interfaz	Cuadrícula, formularios de producto y movimiento	Se agregan el historial de movimientos (Ctrl+H) y el menú de reportes

Ejercitación

Las siguientes actividades fomentan el aprendizaje autónomo e investigativo y se realizan de forma individual y colaborativa.

Actividad individual

- Complete la tercera columna de su tabla SQA («Qué aprendí») con las ideas clave de la fundamentación científica.
- Resuelva la autoevaluación sin mirar el material y luego verifique con la clave que aparece al final de este material.

1. ¿Qué hace un ORM?

a) Diseña formularios

b) Traduce entre objetos del programa y tablas de la base de datos

c) Comprime la base de datos

d) Genera reportes PDF

2. En EF Core, ¿qué método envía a la base los cambios pendientes?

a) Commit()

b) SaveChanges()

c) ExecuteNonQuery()

d) Update()

3. ¿Cuándo se ejecuta realmente una consulta LINQ de EF Core?

a) Al escribir Where

b) Al enumerarla (ToList, First, foreach)

c) Al compilar

d) Al cerrar la aplicación

4. ¿Por qué el total del reporte Excel se escribe como fórmula =SUM(...)?

a) Porque es más bonito

b) Para que se recalcule si el usuario cambia un valor

c) Porque ClosedXML no admite números

d) No importa

5. ¿Qué código de estado HTTP indica éxito?

a) 200

b) 404

c) 500

d) 301

6. ¿Qué excepción aparece típicamente cuando no hay conexión a Internet al usar HttpClient?

a) NullReferenceException

b) HttpRequestException

c) FormatException

d) IndexOutOfRangeException

7. En AgroStock v3 la llamada a Mockaroo se hace con Http.Send. ¿Qué se observa mientras llega la respuesta?

a) La ventana sigue respondiendo con normalidad

b) La ventana no responde un instante, hasta recibir la respuesta

c) La importación se cancela sola

d) La aplicación se cierra

Actividad colaborativa

El trabajo interencuentro se retoma en la BOA 3. Debe realizarse en grupos (salvo las tareas individuales) y entregarse en las fechas indicadas en la BOA.

1. Corregir AgroStock v2 con la retroalimentación del corte 2 (tarea 1).
2. Individual: resolver las 5 consultas LINQ y generar un reporte de una sola categoría (tarea 2).
3. AgroStock v3 final: ORM con EF Core, reportes PDF y Excel, e importación desde Mockaroo (tarea 3).
4. Manual de usuario (máx. 4 páginas) y video de demostración (máx. 3 minutos) (tarea 4).
5. Individual: autoevaluación actitudinal y reflexión sobre el aporte de AgroStock al uso sostenible de recursos (tarea 5).

Práctica de programación

Resuelva de forma individual los siguientes ejercicios. Las 5 consultas LINQ son obligatorias (tarea 2 de la BOA 3); los retos de programación se integran en AgroStock v3. Con la base recién creada por AgroStockContext (datos iniciales sin modificar), los resultados esperados son los indicados entre paréntesis.

Nivel 1 – Básico

1. Devuelva los productos de la categoría «Semillas» ordenados por nombre (3 productos).
2. Cuente los productos con existencia menor que 10 (7 productos).
3. Obtenga el producto de mayor precio con OrderByDescending + FirstOrDefault («Bomba de mochila (20 L)», C$ 1,450.00).
4. Reto de programación 1: agregue un ComboBox de categoría en la barra de la ventana principal y filtre la lista con LINQ: .Where(p => p.CategoriaId == id).

Nivel 2 – Intermedio

1. Calcule el valor total del inventario (C$ 86,381.00).

2. Agrupe por categoría y devuelva cantidad de productos y valor con GroupBy (Fertilizantes 3 · C$ 56,880.00; Semillas 3 · C$ 10,651.00; Herbicidas 5 · C$ 11,600.00; Herramientas 3 · C$ 7,250.00).
3. Reto de programación 2: haga que los reportes PDF y Excel usen la lista filtrada por la búsqueda actual (o por la categoría elegida) en lugar de todos los productos.

Nivel 3 – Avanzado (opcional)

1. Reto de programación 3: ejecute ObtenerProductos dentro de Task.Run con await para que la ventana no se congele durante la llamada a Mockaroo.
2. Reto de programación 4: mueva la clave de Mockaroo a una variable de entorno y léala con Environment.GetEnvironmentVariable.

Entrega

Suba al aula virtual el código de las 5 consultas LINQ con sus resultados y el reporte de una categoría (tarea «Ejercitación E3»), el proyecto AgroStock v3 comprimido sin las carpetas bin, obj ni .vs (con la clave de API que indique su docente), el manual de usuario en PDF con el enlace del video, y complete el formulario de autoevaluación. Si usó ejemplos o IA, indíquelo en el README.

Lista de verificación: marque lo que ya logró.

☐ Configuré AgroStockContext con entidades, restricciones y datos iniciales

☐ Reescribí ProductoRepository con LINQ y SaveChanges, sin SQL a mano

☐ Generé el reporte PDF

☐ Generé el reporte Excel con fórmulas

☐ Obtuve productos de Mockaroo con mi clave de API

☐ Importé el mismo CSV dos veces y el programa informó los repetidos sin fallar

☐ Presenté y defendí AgroStock v3

☐ Entregué manual, video y autoevaluación

C. Cierre y consolidación del aprendizaje

Complementación

El dominio de un ORM, de la generación de reportes y del consumo de servicios web va más allá de memorizar clases o métodos. Reflexione sobre las siguientes preguntas, que conectan lo aprendido con su realidad como futuro(a) ingeniero(a) en sistemas:

Preguntas para reflexionar

- **Conexión con la práctica:** piense en los negocios de su comunidad —un agroservicio, una cooperativa, una pulpería—. ¿Qué tareas semanales, como el reporte de inventario o la carga de una lista de productos, podría automatizar una aplicación como AgroStock?
- **Transferencia del aprendizaje:** ya sabe trabajar con objetos en lugar de SQL manual y obtener datos desde un servicio web. ¿Cómo podría aplicarlo a facturación, planillas o control de ventas?
- **Relación con el eje Madre Tierra:** ¿cómo puede un reporte digital reducir el uso de papel, las pérdidas de insumos vencidos y los viajes innecesarios de los productores? Escriba tres ideas.

Sitios web para ampliar sus conocimientos

Le comparto una selección de recursos en línea, de fuentes confiables, para que pueda seguir perfeccionando sus habilidades:

Recurso	Descripción	Enlace
Microsoft Learn — Introducción a EF Core	Primera aplicación con Entity Framework Core: entidades, DbContext y consultas.	https://learn.microsoft.com/es-es/ef/core/get-started/overview/first-app
QuestPDF — Getting started	Guía de inicio para generar documentos PDF desde C#.	https://www.questpdf.com/quick-start.html
ClosedXML — Documentación	Cómo crear y dar formato a libros de Excel, incluidas fórmulas.	https://docs.closedxml.io/
Microsoft Learn — Solicitudes HTTP con HttpClient	Cómo enviar peticiones HTTP y leer respuestas desde .NET.	https://learn.microsoft.com/es-es/dotnet/fundamentals/networking/http/httpclient

Recurso	Descripción	Enlace
Mockaroo — API Endpoints	Cómo generar datos de prueba con la API: schema, clave y parámetros.	https://mockaroo.com/docs

Investigue los términos de uso y los límites del plan gratuito de Mockaroo antes de usar el servicio en una aplicación real.

Glosario del encuentro

Término	Definición
ORM	Mapeador objeto-relacional: traduce entre objetos del programa y tablas de la base.
DbContext	Clase de EF Core que representa una sesión con la base y rastrea cambios.
DbSet<T>	Colección que representa una tabla y sobre la que se escriben consultas LINQ.
LINQ	Consultas integradas en el lenguaje C# (Where, Select, OrderBy, GroupBy).
SaveChanges	Método que envía a la base todos los cambios pendientes en una transacción.
API REST	Servicio web que expone recursos por URL y se opera con métodos HTTP.
JSON	Formato de texto para intercambiar datos: objetos {}, listas [], texto, números.
Código de estado HTTP	Número que indica el resultado de la petición (200, 404, 500…).
Deserializar	Convertir texto JSON en objetos del programa.
Clave de API	Texto secreto que identifica a su aplicación ante un servicio web; no se publica.
Schema (Mockaroo)	Descripción de los campos de los datos de prueba que genera el servicio.
EnsureCreated	Método de EF Core que crea la base y las tablas si no existen; no actualiza una base ya creada.

Cierre y transferencia del aprendizaje

Generar un reporte o consultar un servicio en Internet no son solo técnicas: son decisiones sobre la confianza que el usuario puede tener en su programa. Un ORM que evita el SQL repetido, reportes con totales verificables y una importación que valida cada fila y avisa de los errores en lugar de cerrarse pueden convertir el inventario de doña Martha en una herramienta confiable, que se puede probar con cientos de productos sin escribirlos a mano.

Así como doña Martha pasó del cuaderno a la pantalla y de la memoria a una base de datos, usted completa en este encuentro el camino de AgroStock: la versión 3 reúne interfaz gráfica, base de datos con ORM, reportes y servicio web.

Conclusiones clave del encuentro

1. Un ORM mapea clases a tablas: se trabaja con objetos y LINQ, y SaveChanges genera el SQL.
2. Un reporte útil tiene encabezado, detalle, totales y pie de página; en Excel los totales son fórmulas.
3. Consumir un servicio web es enviar una petición HTTP y deserializar la respuesta, previendo fallos.
4. AgroStock v3 reúne lo aprendido: interfaz gráfica, base de datos con ORM, reportes PDF y Excel, y un servicio web para importar productos.

Transferencia del aprendizaje

Los conocimientos que ha adquirido son un pilar que trasciende este componente curricular. Los principios de ORM, reportes y servicios web son directamente transferibles a su práctica profesional:

1. Con un ORM, cambiar de SQLite a SQL Server, MySQL o PostgreSQL se reduce, en gran parte, a cambiar el proveedor y la cadena de conexión.
2. La estructura de un reporte (encabezado, detalle, totales y pie) se repite en facturas, planillas y constancias, sea en PDF o en Excel.

3. Consumir un servicio web con HttpClient, deserializar la respuesta y prever fallos le servirá con cualquier API que encuentre en su vida profesional.

“La práctica constante es la clave”. Aplíquese a las consultas del encuentro y a AgroStock v3, y estará construyendo una competencia que le acompañará durante toda su carrera profesional.

Referencias

Ceballos Sierra, F. J. Enciclopedia de Microsoft Visual C#. Editorial RA-MA.

Deitel, H. M. y Deitel, P. J. C# Cómo programar. Editorial Pearson, Prentice Hall.

Microsoft (2024). Guía de escritorio (Windows Forms .NET). Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/desktop/winforms/

Microsoft (2024). Microsoft.Data.Sqlite: información general. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/standard/data/sqlite/

Microsoft (2024). Subprocesamiento administrado y programación asincrónica. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/standard/threading/

Microsoft (2024). Entity Framework Core. Microsoft Learn. https://learn.microsoft.com/es-es/ef/core/

Microsoft (2024). Realización de solicitudes HTTP con la clase HttpClient. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/fundamentals/networking/http/httpclient

SQLite Consortium (2024). SQL As Understood By SQLite. https://www.sqlite.org/lang.html

QuestPDF (2024). Documentación. https://www.questpdf.com/

ClosedXML (2024). Documentación. https://docs.closedxml.io/

Mockaroo (2024). API Endpoints. https://mockaroo.com/docs



¡Universidad
del Pueblo y para el Pueblo!

www. unan.edu.ni